

Cartridge & Cloud - Technical Design Document v0.6

Baseline documental v0.6 - Sprint 16 en curso

VRM Games / Blas Luis Rocha González

01/07/2026

Índice

1. Arquitectura por capas	1
2. Composición	1
3. Sistemas	2
4. Assets representativos	2
5. Problema de la generación procedural	2
6. Arquitectura objetivo de escena	2
7. StoreInitialSceneContext	2
8. Runtime objetivo	2
9. Migración segura	2
10. Input UI	2
11. Testing	3

Proyecto: Cartridge & Cloud

Plataforma: PC / Steam

Motor: Unity 6.3 LTS 6000.3.18f1 / URP 17.3.0

Versión de aplicación: 0.0.17

Baseline documental: v0.6

Último commit técnico validado: 091090c43855b0b26b09abe9335d18b978ac7eab

Último commit observado en main: d54316c771aab2143993e99b9fd58f2f88016568

Estado global: Sprints 0-15 CLOSED / PASS; Sprint 16 IN PROGRESS; Sprint 17 PENDING

1. Arquitectura por capas

- Domain: value objects, agregados e invariantes sin Unity.
- Application: casos de uso y servicios deterministas.
- Infrastructure: persistencia, input, runtime adapters y catálogos.
- Presentation/Runtime: escenas, UI, cámaras, vistas y bridge components.
- Editor: authoring, migraciones, validación e instaladores.
- Tests: EditMode/PlayMode separados.

2. Composición

Bootstrap crea el ApplicationRoot persistente. MainMenu gestiona slots. Store es la escena histórica jugable. StoreInitial será la nueva escena autorada una vez conectada.

3. Sistemas

El vertical slice funcional cubre placement, inventario, proveedores, displays, clientes, shopping, checkout, día, economía, guardado y UI. Los catálogos son ScriptableObjects y los modelos de dominio conservan autoridad.

4. Assets representativos

Los FBX se importan a Art/Models y generan prefabs por familia. El catálogo representativo se referencia desde RuntimeAssetRegistry. Furniture/Product factories sustituyen la presentación conservando roots funcionales.

5. Problema de la generación procedural

Phase1StoreBlockoutBuilder y RepresentativeStoreVisualBuilder intentan derivar visuales desde placeholders técnicos. Bounds, escalas heredadas y ejes del FBX hacen que muros, puertas y mobiliario estático no sean fiables.

6. Arquitectura objetivo de escena

```
StoreInitial
  Systems
  Cameras
  UI
  StoreEnvironment (prefab)
  DynamicFurniture
  DynamicProducts
  Customers
  Debug
```

StoreInitialEnvironment.prefab contiene arquitectura, iluminación, mobiliario inicial, anchors y colliders técnicos. No contiene managers persistentes, HUD, cámara, EventSystem ni save services.

7. StoreInitialSceneContext

Componente con referencias serializadas a placement surface, player spawn, entrance, checkout, receiving, backroom, door controller, initial/dynamic furniture roots, product root, customer spawns y lighting root.

8. Runtime objetivo

En lugar de construir el shell:

1. resolver context;
2. registrar superficie y anchors;
3. registrar mobiliario inicial;
4. configurar puerta;
5. aplicar estado guardado;
6. crear solo mobiliario dinámico;
7. iniciar sistemas de día y clientes.

9. Migración segura

Mantener fallbacks hasta que StoreInitial pase regresión. Desactivar generación procedural mediante configuración explícita, no borrando código durante la primera iteración.

10. Input UI

El controlador de mundo debe retornar antes de raycast cuando EventSystem informa que el puntero está sobre UI.

11. Testing

Los tests deben cubrir resolución de context, ausencia de duplicados, registro de inicial/dinámico, puerta, anchors, UI suppression, save/load y build scene list.